

Contents

Flow Data Collections Specialist Superbadge: sf CLI Workflow	1
Prerequisites	2
What the superbadge wants	2
1. Create a manifest for the flow metadata	2
2. Retrieve the flow files to your local project	3
3. Expected local files	3
4. Open the flow metadata in Neovim	3
5. Build the Opportunity Review flow metadata	4
5.1 Verify the flow identity	4
5.2 Add the formula resource	4
5.3 Verify the Get Records section	4
5.4 Verify the collection variables	5
5.5 Verify the stage filter choice resources	5
5.6 Verify the screen and Data Table	5
5.7 Verify the decision and filter path	6
5.8 Verify alphabetical sorting	6
5.9 Fast verification search	6
6. Build the Opportunity Updater flow metadata	6
6.1 Verify the input collection variable	6
6.2 Verify the loop	7
6.3 Verify the update screen	7
6.4 Verify assignments after the screen	7
6.5 Verify the final Update Records step	7
6.6 Fast verification search	7
7. Deploy the flow metadata to your org	8
8. Useful verification commands	8
9. Fast Neovim workflow	8
10. Minimal command list	9
11. Expected local files	9

Flow Data Collections Specialist Superbadge: sf CLI Workflow

This README translates the Trailhead **Flow Data Collections Specialist Superbadge** into a Neovim + Salesforce CLI workflow that stays as close as possible to a terminal-first TUI flow. Salesforce stores flows as Flow metadata in a Salesforce DX project, and the CLI can retrieve and deploy those flow files so you can inspect and edit them locally instead of relying on the web UI for every iteration.

The superbadge has two required automations: a screen flow named **Opportunity Review** that displays and filters recent opportunities, and a screen flow named **Opportunity Updater** that collects user edits and

performs efficient bulk updates on the selected opportunities.

Prerequisites

Make sure Salesforce CLI is installed, your org is authenticated, and you are inside a Salesforce DX project directory.

Recommended setup commands:

```
sf org list
sf config set target-org=matthew.porter@playful-impala-wigh6g.com
mkdir -p force-app/main/default/flows
mkdir -p manifest

sf org list shows your authorized orgs, and sf config set target-org=...
lets later commands use your default org automatically.
```

What the superbadge wants

The first challenge requires a screen flow named **Opportunity Review** with API name **Opportunity_Review** that gets opportunities created in the last 30 days, displays them in a Data Table, and lets the user filter the displayed collection by opportunity stage. The second challenge requires the **Opportunity Updater** screen flow to accept an input collection named **Selected_Opportunities**, loop through the opportunities, collect edits from a screen, store the changes in a collection, and update them with the minimum number of DML statements.

1. Create a manifest for the flow metadata

Create a `package.xml` manifest so you can retrieve and deploy the two required flows repeatedly from the terminal.

Open the manifest:

```
nvim manifest/package.xml
```

Paste this XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<Package xmlns="http://soap.sforce.com/2006/04/metadata">
  <types>
    <members>Opportunity_Review</members>
    <members>Opportunity_Updater</members>
    <name>Flow</name>
  </types>
  <version>61.0</version>
</Package>
```

Flow is a Metadata API type, so you can retrieve and deploy it through a standard manifest or by naming each `Flow:<API_Name>` member directly.

2. Retrieve the flow files to your local project

Retrieve the flow metadata so you have local XML files to inspect and edit in Neovim.

If your CLI supports direct retrieve syntax:

```
sf retrieve metadata --manifest manifest/package.xml --target-org matthew.porter@playful-imp
```

If you prefer project retrieval syntax:

```
sf project retrieve start --manifest manifest/package.xml --target-org matthew.porter@playfu
```

Salesforce DX stores flow source in `force-app/main/default/flows/*.flow-meta.xml`, which is the file format you will review and deploy for this workflow.

3. Expected local files

After retrieval, your project should have these files or their currently-versioned equivalents:

```
force-app/main/default/flows/Opportunity_Review.flow-meta.xml
force-app/main/default/flows/Opportunity_Updater.flow-meta.xml
manifest/package.xml
```

If the files do not exist yet because the flows have never been created in the org, you can still author them locally, but the safer workflow is to retrieve any starter metadata first and then edit the existing XML structure.

4. Open the flow metadata in Neovim

Open both flow files in Neovim:

```
nvim force-app/main/default/flows/Opportunity_Review.flow-meta.xml \
     force-app/main/default/flows/Opportunity_Updater.flow-meta.xml
```

Because Flow metadata can be verbose, it helps to use XML formatting and search by element type instead of trying to reason about the entire file at once.

Useful searches:

```
/fullName
/processType
/variables
/formulas
/recordLookups
/assignments
/decisions
/screens
/loops
/recordUpdates
```

Salesforce's Flow metadata model includes the full flow structure, including screens, record operations, assignments, loops, and decisions, all encoded in the `.flow-meta.xml` file.[cite:289]

5. Build the Opportunity Review flow metadata

The Opportunity Review flow must be a screen flow that retrieves opportunities created in the last 30 days, shows them in a Data Table, allows filtering by stage, and maintains alphabetical sorting by opportunity Name when the displayed subset changes.

5.1 Verify the flow identity

In `force-app/main/default/flows/Opportunity_Review.flow-meta.xml`, verify or create the flow identity values:

- `<fullName>Opportunity_Review</fullName>`
- A label or master label of Opportunity Review
- A screen-flow process type

5.2 Add the formula resource

Look for the `<formulas>` section and make sure it contains a formula resource named `Today_Minus_30` with `DateTime` type and expression `DATETIMEVALUE(Today() - 30)`.

Example structure:

```
<formulas>
  <name>Today_Minus_30</name>
  <dataType>DateTime</dataType>
  <expression>DATETIMEVALUE(Today() - 30)</expression>
</formulas>
```

5.3 Verify the Get Records section

Look for the `<recordLookups>` section that retrieves Opportunity records. It should filter on `CreatedDate >= {!Today_Minus_30}` and store all matching records in a collection instead of stopping after the first result.

Things to verify:

- object type is Opportunity
- filter field is CreatedDate
- filter value points to Today_Minus_30
- the result is a record collection

5.4 Verify the collection variables

The flow should have at least one collection variable for the current display set and ideally another collection for the original unfiltered opportunities. Inspect the `<variables>` section for collection resources with object type `Opportunity`.

Typical pattern:

```
<variables>
  <name>Displayed_Opportunities</name>
  <dataType>SObject</dataType>
  <isCollection>true</isCollection>
  <isInput>false</isInput>
  <isOutput>false</isOutput>
  <objectType>Opportunity</objectType>
</variables>
```

5.5 Verify the stage filter choice resources

The screen needs two kinds of choice resources:

- a picklist-backed choice set for Opportunity `StageName`
- a manual text choice whose value is `All` so the user can reset the filter.

Look for a manual choice that resembles:

```
<choices>
  <name>View_All</name>
  <dataType>String</dataType>
  <value>
    <stringValue>All</stringValue>
  </value>
</choices>
```

5.6 Verify the screen and Data Table

Inside the `<screens>` section, find the screen node that contains the stage selection control and the Data Table component. The Data Table should point to the display collection and expose the required fields

- Name
- StageName
- Amount
- ExpectedRevenue
- Description

When inspecting the screen XML, verify that:

- the stage selector stores an output value used later by the decision,
- the Data Table source collection references the current display collection,
- the displayed columns match the challenge requirements exactly.

5.7 Verify the decision and filter path

Look for a `<decisions>` block that checks whether the selected stage does not equal All. The non-All path should route to a filter element named `Filter_for_StageName` or an equivalent collection-processing node that filters the records where `StageName` matches the selected value.

After filtering, inspect the next `<assignments>` block and verify that it copies the filtered results back into the displayed collection for the next screen render.

5.8 Verify alphabetical sorting

The challenge requires the opportunities to be sorted alphabetically by Name not only on initial load, but also after the displayed subset changes. Inspect the collection-processing or sort-related section and verify that Name-based ascending order is applied after filtering as well as at initial collection setup.

5.9 Fast verification search

Use ripgrep to quickly confirm the important names are present:

```
rg -n "Opportunity_Review|Today_Minus_30|Displayed_Opportunities|StageName|ExpectedRevenue|I
force-app/main/default/flows/Opportunity_Review.flow-meta.xml
```

6. Build the Opportunity Updater flow metadata

The Opportunity Updater flow must accept an input collection of selected opportunities, iterate that collection, collect edits from the screen, store the updated loop records in a second collection, and perform one final Update Records operation after the loop completes.

6.1 Verify the input collection variable

In `force-app/main/default/flows/Opportunity_Updater.flow-meta.xml`, inspect the `<variables>` section and verify a record collection variable named `Selected_Opportunities` exists, is an input, is a collection, and uses object type `Opportunity`.

Pattern to look for:

```
<variables>
  <name>Selected_Opportunities</name>
  <dataType>SObject</dataType>
  <isCollection>true</isCollection>
  <isInput>true</isInput>
  <isOutput>false</isOutput>
  <objectType>Opportunity</objectType>
</variables>
```

6.2 Verify the loop

Find the `<loops>` section and verify the loop iterates over `Selected_Opportunities`. The loop should expose a current item variable, often something like `Current_Opportunity`, which the later assignments modify.

6.3 Verify the update screen

Inside the `<screens>` section, find the `Update Opportunity` screen and inspect its fields. The screen must expose editable values for:

- Name
- Account
- Close Date
- Description

Those screen outputs should later be referenced in assignments that write the screen values back into the loop variable.

6.4 Verify assignments after the screen

Inspect the `<assignments>` section following the screen and confirm that the flow stores the screen outputs back into the current loop record. Then verify there is another assignment that adds the modified loop record into an update collection such as `Opportunities_To_Update`.

Things to inspect in the XML:

- assignment target points to a field on the current loop record,
- assignment source points to a screen output,
- add-to-collection assignment targets a collection variable for later bulk update.

6.5 Verify the final Update Records step

Look for the `<recordUpdates>` section that runs after the loop. This should update the collection of modified opportunities in a single step rather than performing DML inside the loop, which is the efficiency requirement called out in the superbadge.

Verify that:

- the update step references the collection variable,
- the update node is outside the loop body,
- there is no record update node inside the loop path.

6.6 Fast verification search

```
rg -n "Opportunity_Updater|Selected_Opportunities|CloseDate|Description|Account|recordUpdates" force-app/main/default/flows/Opportunity_Updater.flow-meta.xml
```

7. Deploy the flow metadata to your org

Once both flow files are edited, deploy them with Salesforce CLI.

Deploy by manifest:

```
sf project deploy start --manifest manifest/package.xml --target-org matthew.porter@playful-
```

Or deploy the individual flow members:

```
sf project deploy start --metadata Flow:Opportunity_Review --target-org matthew.porter@playful-
sf project deploy start --metadata Flow:Opportunity_Updater --target-org matthew.porter@playful-
```

The `project deploy start` command deploys local project metadata back to the org, which is the key step before clicking the Trailhead checker.

8. Useful verification commands

To confirm the files are present locally:

```
ls force-app/main/default/flows/Opportunity_Review.flow-meta.xml
ls force-app/main/default/flows/Opportunity_Updater.flow-meta.xml
ls manifest/package.xml
```

To inspect the local flow files with formatting:

```
xmllint --format force-app/main/default/flows/Opportunity_Review.flow-meta.xml | less
xmllint --format force-app/main/default/flows/Opportunity_Updater.flow-meta.xml | less
```

To compare changes before deploy:

```
git diff -- force-app/main/default/flows/Opportunity_Review.flow-meta.xml
git diff -- force-app/main/default/flows/Opportunity_Updater.flow-meta.xml
```

To retrieve the latest org copy again if you need to reset or inspect drift:

```
sf project retrieve start --manifest manifest/package.xml --target-org matthew.porter@playful-
```

9. Fast Neovim workflow

1. Retrieve the flow metadata.
2. Edit `force-app/main/default/flows/Opportunity_Review.flow-meta.xml`.
3. Edit `force-app/main/default/flows/Opportunity_Updater.flow-meta.xml`.
4. Save both files.
5. Run your deploy mapping or execute `sf project deploy start --manifest manifest/package.xml`.
6. Review deploy output in the terminal.
7. Click **Check Challenge** in Trailhead.

If you already added Neovim commands like `:SfRetrieve` and `:SfDeploy`, the loop becomes:


```
:edit force-app/main/default/flows/Opportunity_Review.flow-meta.xml
:write
:edit force-app/main/default/flows/Opportunity_Updater.flow-meta.xml
:write
:SfDeploy
```

10. Minimal command list

If you only want the exact commands to complete the superbadge workflow, use this sequence:

```
sf org list
sf config set target-org=matthew.porter@playful-impala-wigh6g.com
mkdir -p force-app/main/default/flows manifest
nvim manifest/package.xml
sf project retrieve start --manifest manifest/package.xml --target-org matthew.porter@playful-impala-wigh6g.com
nvim force-app/main/default/flows/Opportunity_Review.flow-meta.xml
nvim force-app/main/default/flows/Opportunity_Updater.flow-meta.xml
sf project deploy start --manifest manifest/package.xml --target-org matthew.porter@playful-impala-wigh6g.com
```

11. Expected local files

Your project should end up with at least these files:

```
manifest/package.xml
force-app/main/default/flows/Opportunity_Review.flow-meta.xml
force-app/main/default/flows/Opportunity_Updater.flow-meta.xml
```

That structure matches Salesforce DX source format for local Flow metadata editing and CLI deployment.